

COMP40771 – Lab L5: Reinforcement Learning

Lab 5 was designed for improving your understanding about Reinforcement Learning.

Goals:

- Build intuition for MDPs
- Implement MC, TD(0), Q-learning, SARSA
- Compare exploration effects
- (Optional) Linear function approximation

Sections marked **TODO** require completion.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
```

Part 1 - Gridworld environment (episodic)

Establish a small, fully observable MDP that students can inspect and use as the testbed for all algorithms. Complete `GridworldEnv.step(action)` so that actions move the agent with boundary handling, returns (next_state, reward, done), adds a step penalty each move, gives +1 at the goal (and terminates), and applies trap penalties when entered.

You will see “Start state: ...” and displays a 5×5 grid with A at the start, G at the goal, and T at trap locations. Stepping the environment (later parts) will no longer error and episodes will terminate when reaching G.

Define:

- Actions: 0=Up, 1=Right, 2=Down, 3=Left.
- Goal gives +1 and terminates. Step penalty encourages shorter paths. Optional traps add negative reward.

```
# TODO: Complete the GridworldEnv.step() method.

class GridworldEnv:
    def __init__(self, width=5, height=5, start=(0,0), goal=(4,4),
traps=None, step_penalty=-0.01):
        self.width = width
        self.height = height
        self.start = start
        self.goal = goal
        self.traps = traps if traps is not None else {(2,2): -1.0}
        self.step_penalty = step_penalty
        self.n_actions = 4
        self.n_states = width * height
```

```

        self.reset()

    def _to_state(self, pos):
        x, y = pos
        return y * self.width + x

    def reset(self):
        self.pos = self.start
        return self._to_state(self.pos)

    def step(self, action):
        """Return (next_state, reward, done)."""
        # TODO: update self.pos with boundary handling
        # TODO: reward = step_penalty + goal_reward/trap_reward where
        applicable
        # TODO: done=True if goal reached
        raise NotImplementedError

    def render(self):
        grid = np.full((self.height, self.width), '.', dtype=object)
        gx, gy = self.goal
        grid[gy, gx] = 'G'
        for (tx, ty), _r in self.traps.items():
            grid[ty, tx] = 'T'
        ax, ay = self.pos
        grid[ay, ax] = 'A'
        print("\n".join(" ".join(row) for row in grid))

env = GridworldEnv()
s = env.reset()
print('Start state:', s)
env.render()

```

Helpers (ϵ -greedy, running episodes, plotting)

Provide reusable utilities for exploration, episode generation, and plotting learning curves.

Implement `epsilon_greedy(Q, state, epsilon, n_actions)` so that with probability ϵ it returns a random action, otherwise it returns the greedy action (argmax over Q).

You will see Q-learning and SARSA sections will run without exceptions. Changing ϵ will visibly affect learning curves (more exploration initially tends to give noisier returns, but can improve eventual performance).

```

def epsilon_greedy(Q, state, epsilon, n_actions):
    """Return an action using  $\epsilon$ -greedy exploration."""
    # TODO: with probab epsilon pick random; else pick argmax
    raise NotImplementedError

def run_episode(env, policy_fn, max_steps=200):

```

```

    """Generate an episode: list of (s, a, r)."""
    s = env.reset()
    traj = []
    for _ in range(max_steps):
        a = policy_fn(s)
        s2, r, done = env.step(a)
        traj.append((s, a, r))
        s = s2
        if done:
            break
    return traj

def plot_learning_curve(returns, title):
    plt.figure()
    plt.plot(returns)
    plt.xlabel('Episode')
    plt.ylabel('Return')
    plt.title(title)
    plt.show()

```

Part 2 — First-visit Monte Carlo prediction ($V(s)$)

Learn state values from complete episodes without bootstrapping; highlight high variance and episode-based learning.

Implement first-visit MC prediction inside `mc_prediction(...)`.

- generate an episode under the random policy,
- compute the return G backwards through the episode,
- apply the first-visit rule (update V only the first time a state appears in that episode),
- maintain running averages using `returns_sum` and `returns_count`. You will see the cell prints a vector sample such as MC $V(s)$ sample: [...]. Values nearer the goal will typically be higher (less negative due to step penalties, more likely to reach +1), while values in/near traps will be lower. Runs may look noisy because the policy is random and MC has high variance.

```

# TODO: Implement first-visit MC prediction

def mc_prediction(env, num_episodes=2000, gamma=0.99):
    V = np.zeros(env.n_states)
    returns_sum = np.zeros(env.n_states)
    returns_count = np.zeros(env.n_states)

    def random_policy(_s):
        return np.random.randint(env.n_actions)

    for _ in range(num_episodes):
        episode = run_episode(env, random_policy)
        # TODO: compute return G backwards and do first-visit updates

```

```

        raise NotImplementedError

    return V

V_mc = mc_prediction(env)
print('MC V(s) sample:', V_mc[:10])

```

Part 3 - TD(0) prediction (V(s))

Learn state values online using bootstrapping; compare behaviour to Monte Carlo.

Student TODO: Implement the TD(0) update in `td0_prediction(...)`:

$$V(s) \leftarrow V(s) + \alpha(r + \gamma V(s') - V(s))$$

You must compute the TD error and update $V[s]$ each step.

You will see a learning curve plot titled “TD(0) – random policy” appears. Returns under a random policy will remain low/variable, but TD learning runs efficiently and does not require episode-end returns. Values can stabilise sooner than MC in many settings.

```

# TODO: Implement TD(0) prediction

def td0_prediction(env, num_episodes=2000, alpha=0.1, gamma=0.99):
    V = np.zeros(env.n_states)

    def random_policy(_s):
        return np.random.randint(env.n_actions)

    returns = []
    for _ in range(num_episodes):
        s = env.reset()
        G_ep = 0.0
        for _ in range(200):
            a = random_policy(s)
            s2, r, done = env.step(a)
            # TODO: TD update
            raise NotImplementedError
            s = s2
            G_ep += r
            if done:
                break
        returns.append(G_ep)
    return V, returns

V_td, ret_td = td0_prediction(env)
plot_learning_curve(ret_td, 'TD(0) – random policy')

```

Part 4 — Control: Q-learning and SARSA

Move from prediction to control; learn an action-value function and derive a greedy policy. Compare off-policy vs on-policy learning.

Student TODO (Q-learning): Implement the Q-learning update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

You must compute the greedy next-state value with `np.max(Q[s2])`.

Implement the SARSA update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a))$$

You must use the actual next action `a2` selected by ϵ -greedy.

You will see:

- Two plots: “Q-learning” and “SARSA” (episode returns). Over training, returns should improve as agents reach the goal more reliably.
- Q-learning often improves faster but can show more instability depending on ϵ and traps. SARSA often looks more conservative.
- No `NotImplementedError` occurs once the TODOs are complete.

TODO: *Implement Q-learning and SARSA updates*

```
def q_learning(env, num_episodes=3000, alpha=0.1, gamma=0.99,
               epsilon=0.1):
    Q = np.zeros((env.n_states, env.n_actions))
    returns = []
    for _ in range(num_episodes):
        s = env.reset()
        G_ep = 0.0
        for _ in range(200):
            a = epsilon_greedy(Q, s, epsilon, env.n_actions)
            s2, r, done = env.step(a)
            # TODO: Q-learning update
            raise NotImplementedError
            s = s2
            G_ep += r
            if done:
                break
        returns.append(G_ep)
    return Q, returns

def sarsa(env, num_episodes=3000, alpha=0.1, gamma=0.99, epsilon=0.1):
    Q = np.zeros((env.n_states, env.n_actions))
    returns = []
    for _ in range(num_episodes):
        s = env.reset()
```

```

a = epsilon_greedy(Q, s, epsilon, env.n_actions)
G_ep = 0.0
for _ in range(200):
    s2, r, done = env.step(a)
    a2 = epsilon_greedy(Q, s2, epsilon, env.n_actions)
    # TODO: SARSA update
    raise NotImplementedError
    s, a = s2, a2
    G_ep += r
    if done:
        break
    returns.append(G_ep)
return Q, returns

Q_q, ret_q = q_learning(env)
Q_s, ret_s = sarsa(env)

plot_learning_curve(ret_q, 'Q-learning')
plot_learning_curve(ret_s, 'SARSA')

```

Visualise greedy policy (arrows)

Convert learned Q-tables into an interpretable policy map.

This section depends on earlier TODO completion.

You will see two printed 5×5 grids of arrows (↑ → ↓ ←), plus G and T. Arrows should generally point toward the goal while avoiding traps. Q-learning and SARSA policies may differ slightly near traps or risky routes.

```

ARROWS = {0:'↑', 1:'→', 2:'↓', 3:'←'}

def show_greedy_policy(env, Q):
    grid = np.full((env.height, env.width), '.', dtype=object)
    for y in range(env.height):
        for x in range(env.width):
            if (x,y) == env.goal:
                grid[y,x] = 'G'
            elif (x,y) in env.traps:
                grid[y,x] = 'T'
            else:
                s = y * env.width + x
                grid[y,x] = ARROWS[int(np.argmax(Q[s]))]
    print('\n'.join(' '.join(row) for row in grid))

print('Greedy policy from Q-learning:')
show_greedy_policy(env, Q_q)
print('\nGreedy policy from SARSA:')
show_greedy_policy(env, Q_s)

```

Part 5 (Optional) — Linear function approximation for $Q(s,a)$

Introduce scalability beyond tabular methods; demonstrate a simple approximator and the trade-offs.

Implement `phi(env, s, a)` to return a feature vector by concatenating:

- one-hot encoding of the state (length `n_states`), and
- one-hot encoding of the action (length `n_actions`).

Student TODO (learning rule): Implement semi-gradient linear Q-learning update on weights `w`:

- compute prediction: $Q^{(s,a)} = \phi(s,a)^T w$
- compute target: $r + \gamma \max_i Q^{(s,i)}$
- update: $w \leftarrow w + \alpha (target - pred) \phi(s,a)$

You will see a plot titled “Linear Q-learning” appears. Learning may be slower or less stable than tabular Q-learning, but it demonstrates how the same RL objective can be optimised with parameter vectors rather than lookup tables.

```
# TODO: Implement feature mapping and semi-gradient linear Q-learning

def phi(env, s, a):
    # TODO: concatenate one-hot(state) and one-hot(action)
    raise NotImplementedError

def q_linear(env, num_episodes=5000, alpha=0.05, gamma=0.99,
epsilon=0.1):
    d = env.n_states + env.n_actions
    w = np.zeros(d)
    returns = []

    for _ in range(num_episodes):
        s = env.reset()
        G_ep = 0.0
        for _ in range(200):
            q_vals = np.array([phi(env, s, a) @ w for a in
range(env.n_actions)])
            a = np.random.randint(env.n_actions) if np.random.rand() <
epsilon else int(np.argmax(q_vals))

            s2, r, done = env.step(a)

            # TODO: semi-gradient update on w
            raise NotImplementedError

            s = s2
            G_ep += r
```

```
        if done:
            break
        returns.append(G_ep)

    return w, returns

w_lin, ret_lin = q_linear(env)
plot_learning_curve(ret_lin, 'Linear Q-learning')
```

Wrap-up questions

1. What differences do you observe between MC and TD(0)?
2. Under ϵ -greedy exploration, how do Q-learning and SARSA differ?
3. What trade-offs arise with function approximation?